

Cryptography

3 – Authentication and hash functions

Gabriel Chênevert

September 29, 2025

JUNIA ISEN

Today

User authentication

Hash function design

Message authentication

User authentication

Applications often need to ask users (or devices...) to identify themselves in order to know how to behave.



id : Alice



id : Bob



id : Alice

gnark gnark

Authentication factors

Obviously such an input needs to be **authenticated** (confirmed).

Authentication methods rely on different **factors** such as:

- something the user *knows*,
- something the user *has*,
- something the user *is* (or a way they *behaves*).

Ultimately they all boil down to have the user provide a certain sequence of bits that we assume only they are able to provide.

Password authentication

Upon registration, every user provides (or is assigned) a password.



id : Alice

pw : Ii(H48s



id : Bob

pw : secret

...

Naive implementation

All valid pairs (id, pw) are stored by the service provider Sammy.



When a pair (id, pw') is received, Sammy checks whether

$$pw' = pw.$$

Problem

An attacker with read access recovers all the passwords.

(Equivalently: need absolute trust in Sammy!)

Storing encrypted versions $E(k, pw)$ seems better...

...is it ? (hint: not that much)

NB: *sending* encrypted passwords on the communication channel is certainly a good idea, though

Solution

Use *one-way (lossy) encryption*

i.e. a **hash function**

$$H : \{0, 1\}^* \longrightarrow \{0, 1\}^n.$$

Examples:

MD5 (deprecated), SHA-1 (deprecated),

SHA-2, SHA-3, BLAKE, BLAKE2, BLAKE3, Whirlpool, ...

Usage

A hash function turns everything into a fixed-length bit sequence.

```
from hashlib import md5, sha1, sha256
```

```
message = b"A hash function turns arbitrary inputs into digests of fixed size"
```

```
print("msg :", message)
print()
print("MD5 :", md5(message).hexdigest())    # 128 bits
print("SHA1:", sha1(message).hexdigest())   # 160 bits
print("SHA2:", sha256(message).hexdigest()) # 256 bits
```

```
msg : b'A hash function turns arbitrary inputs into digests of fixed size'
```

```
MD5 : faf72bc60a7d6011de46eb3d275c7335
```

```
SHA1: 1571fbba39bc28b1cf77ad88f55db6000ce6d980
```

```
SHA2: 42830b8a9452e04b8909d902ebd8b7fe9d6de88931ffd77a66c6b68da64c3d7b
```

Better password management

Sammy stores, for every valid user, a hash of their password:

$$(\text{id}, h) \quad \text{with} \quad h = H(\text{pw}).$$

Authentication:

Upon reception of (id, pw') , Sammy checks if

$$H(\text{pw}') = h.$$

Requirement

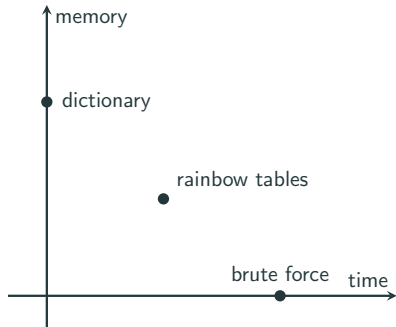
The hash function should be **preimage resistant**:

given h , it must be computationally hard to find m such that

$$H(m) = h.$$

Attacks:

- brute force
- dictionary (precomputed)
- **rainbow tables** (space-time tradeoff)



Improvements

- **Salting:** store $(id, s, H(s \parallel pw))$ where s is random *salt*
- **Key stretching:** more generally, use a *key derivation function* to generate

$$k = K(s, pw) \quad \text{and store} \quad (id, s, k)$$

where K is made *deliberately slow*

Examples: Argon2, PBKDF2, Bcrypt, scrypt, ...

$\Rightarrow$ in practice: **always** use such a specialized *key derivation function* (KDF)

Today

User authentication

Hash function design

Message authentication

Cryptographic hash functions

Hash functions are useful for many things:

- id generation
- hash tables
- pattern detection
- serialization
- ...

but certain specific properties are required for their use in cryptography.

Requirements

- **determinism:** $m = m' \implies H(m) = H(m')$
- **uniformity:** every hash occurs with probability $1/2^n$
- **avalanche:** $m \approx m', m \neq m' \implies H(m) \not\approx H(m')$
(exactly the opposite of **continuity**)

Things that should be hard

- given h , find m such that $H(m) = h$
(**preimage resistance**)
- given m , find $m' \neq m$ such that $H(m') = H(m)$
(**second preimage resistance**)
- find $m \neq m'$ such that $H(m) = H(m')$
(**collision resistance**)

↑
difficulty

A textbook case: the story of SHA-1

- 1995: **Secure Hash Algorithm 1** standardized by NIST
- 2005: first "theoretical" collision attacks published
- 2010: collision complexity brought down to roughly 2^{60}
Estimated cost of attack: 3 M\$
- 2015: "the SHAppening" first practical attack demonstration
Estimated cost of attack: 100 k\$
- 2017: "SHAttered" first public collision
- 2019: Improved chosen prefix attack

The birthday problem (or "paradox")

- generating $N > 2^n$ hashes $\implies$ *certain* collision
- if N values are generated uniformly at random, the *probability* of a collision is

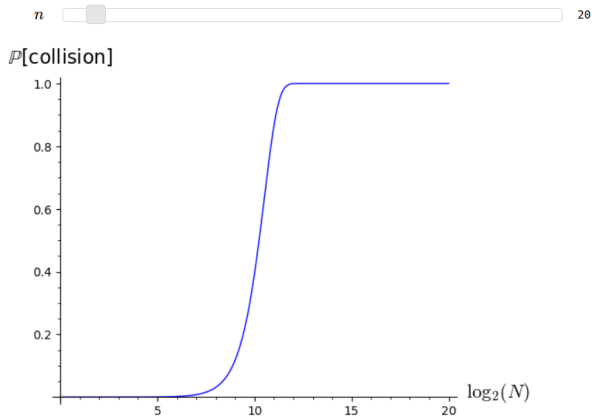
$$p = 1 - \prod_{k=0}^{N-1} \left(1 - \frac{k}{2^n}\right) \approx 1 - e^{-\frac{1}{2^n} \binom{N}{2}} \approx 1 - e^{-\frac{1}{2^{n+1}} N^2}$$

Example: The probability that 28 randomly chosen persons share a birthday is

$$\approx 1 - e^{-\frac{1}{365} \binom{28}{2}} \approx 65\%$$

NB: non-uniformity in the distribution of values only make collisions *more* probable

Collision probability as function of hash length



Birthday attack

One can show that the *average* number of values to be generated before a collision is found is approximately

$$\sqrt{\pi 2^{n-1}} \approx 1.25 \times 2^{\frac{n}{2}}.$$

Hence: a n -bit hash function provides $\leq \frac{n}{2}$ bits of security.

$\implies$ hashes need to be at least 256 bits long to provide 128 bits of security.

Pearson hash

An *insecure* construction

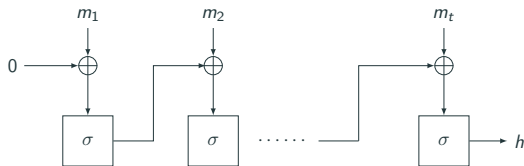
Divide the message m into ℓ -bit blocks $m = m_1 \parallel m_2 \parallel \dots \parallel m_t$

and choose a permutation σ of $\{0, 1\}^\ell = \llbracket 0, 2^\ell \llbracket$.

$h = 0$

for m_i in m :

$$h = \sigma(h \oplus m_i)$$



Nice, but specifying σ takes $\ell \cdot 2^\ell$ memory ...

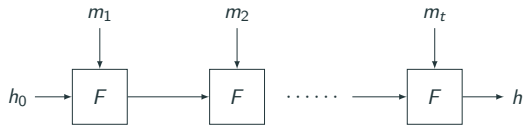
Merkle-Damgård construction

Reuses the idea of Pearson hashing.

$$h = h_0$$

for m_i in m :

$$h = F(h, m_i)$$



where the *compression function* F is typically a simple operation iterated r times on the internal state (size s , divided into w -bit words)

Famous cryptographic hash functions

name	published	deprecated	n	ℓ	s	w	r
MD5	1991	2000	128	512	512	32	64
SHA-1	1995	2005	160	512	160	32	64
SHA-2	2001	–	256 (224)	512	256	32	64
			512 (448)	1024	512	64	80
BLAKE2	2008	–	256 (224)	64	128	32	14
			512 (448)	128	256	64	16
SHA-3	2012	–		...			

SHA-3 (Keccak, ST Microelectronics)

Sponge construction

$$(R, C) = (R_0, C_0)$$

// absorption

for m_i in m :

$$(R, C) = F(R \oplus m_i, C)$$

// then some more drying

eventually output R

Allows for certain freedom in choice of parameters

e.g. SHA3-224, SHA3-256, SHA3-384, SHA3-512, ...

Today

User authentication

Hash function design

Message authentication

Hash as checksum

Hash functions can be used to verify message integrity.

Alice: appends to a message m its hash $h = H(m)$.

Bob: verifies upon reception of (m, h) that $h = H(m)$.

(If not: transmission problem detected)

Example



$m = \text{You owe me \$10}$

$h = \text{c7b12b33fdd17399}$

$m_{\text{received}} = \text{You owe me \$10}$

$h_{\text{received}} = \text{c7b12b33fdd17399}$

$h_{\text{computed}} = \text{c7b12b33fdd17399}$

Ok !

Example (cont'd)



$m = \text{You owe me \$10}$

$h = \text{c7b12b33fdd17399}$

$m_{\text{received}} = \text{You owe me \$100}$

$h_{\text{received}} = \text{c7b12b33fdd17399}$

$h_{\text{computed}} = \text{08821af9be531f29}$

Error !

But also...



$m = \text{You owe me \$100}$

$h = 08821af9be531f29$

$m_{\text{received}} = \text{You owe me \$100}$

$h_{\text{received}} = 08821af9be531f29$

$h_{\text{computed}} = 08821af9be531f29$

Ok ! ...

Problem

Even if H cannot be manipulated ...

anybody can compute a valid hash!

Double-edged sword:

- falsification
- repudiation

$\implies$ no authentication at all

Idea: encrypt the hash

Alice: appends to m its encrypted hash $h = E(k, H(m))$

Bob: upon reception of (m, h) , checks whether $H(m) = D(k, h)$

Problem: since $H(m)$ and h are public, the secret key k is exposed...

Message authentication codes

Definition

A **MAC** consists of a *tag* function $\mathcal{K} \times \mathcal{M} \rightarrow \mathcal{T}$ as well as a *verification algorithm* that decides whether a particular MAC is valid for a given message.

- **Correctness:** every generated MAC should be valid
- **Forgery resistance:** no one should be able to create a valid pair (m, τ) without knowing the key.

From a hash function

Standard construction:

$$\text{HMAC}(k, m) := H((k \oplus \text{opad}) \parallel H((k \oplus \text{ipad}) \parallel m))$$

where ipad and opad are two (constant) masks

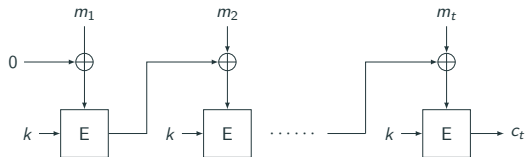
Alice: appends to m its tag $\tau = \text{HMAC}(k, m)$

Bob: verifies upon reception of (m, τ) whether $\tau = \text{HMAC}(k, m)$

From a block cipher

Idea: Encrypt $m = m_1 \parallel \dots \parallel m_t$ in CBC-mode with $IV = 0$.

$$\text{CBC-MAC}(k, m) := c_t$$



+ additional precautions to prevent *extension attacks*

Authenticated encryption

Given a secure cipher + a secure MAC:

- **encrypt then MAC**: always ok
- encrypt and MAC: weakens encryption
- MAC then encrypt: ok in some cases

Never reuse the same key for different purposes!

End remarks

- AE provides confidentiality, authentication, integrity, non-repudiation
- modern approach is to provide AE as a single primitive
- examples: OCB, EAX, EtM, GCM, CCM modes
- AE does not prevent *replay attacks* by itself

⇒ Authenticated Encryption with Associated Data (AEAD) as IV should be used.

Message is encrypted, message + AD is authenticated